

Figure 11.1: The string "Hello" has length 5 and valid indexes 0–4. `substring(1, 4)` keeps indexes 1, 2, 3: the start is *included* and the end is *excluded*.

From Python to Java: slicing and indexing

Python: `s[1:4]`, `s[1]`, `len(s)`. Java: `s.substring(1, 4)`, `s.charAt(1)`, `s.length()`. Both start counting at 0 and exclude the end of the slice. Differences: Java has *no square-bracket indexing for strings*, no negative indexes, and `length` needs parentheses: `s.length()`.

11.1.3 The most important methods

Method	Returns
<code>int length()</code>	the number of characters
<code>String substring(int from)</code>	the characters from index <code>from</code> to the end
<code>String substring(int from, int to)</code>	the characters from <code>from</code> up to but not including to
<code>int indexOf(String str)</code>	the index where <code>str</code> first occurs, or <code>-1</code> if it does not
<code>boolean equals(String other)</code>	<code>true</code> if same characters (case-sensitive)
<code>int compareTo(String other)</code>	negative, 0 or positive: alphabetical order
<code>char charAt(int i)</code>	the character at index <code>i</code>
<code>String concat(String str)</code>	the two strings joined (same as <code>+</code>)
<code>String toUpperCase()</code> , <code>String toLowerCase()</code>	a copy with all letters converted
<code>String replace(char a, char b)</code>	a copy with every <code>a</code> replaced by <code>b</code>
<code>String trim()</code>	a copy without leading and trailing spaces

AP Exam: the AP String subset

The six methods above the line — `length`, both `substring` forms, `indexOf(String)`, `equals` and `compareTo` — are on the AP Quick Reference. (Newer exams also list `charAt`-style access through `substring(i, i + 1)`.) Learn them cold; the others are useful in real programs.

StringBasics.java

```

1 public class StringBasics {
2     public static void main(String[] args) {
3         String title = "Java Software Solutions";
4         System.out.println("length : " + title.length());
5         System.out.println("substring(5) : " + title.substring(5));
6         System.out.println("substring(0, 4) : " + title.substring(0, 4));
7         System.out.println("indexOf(\"Soft\") : " + title.indexOf("Soft"));
8         System.out.println("indexOf(\"zzz\") : " + title.indexOf("zzz"));
9         System.out.println("indexOf(\"o\") : " + title.indexOf("o"));
10        System.out.println("equals : " + title.equals("Java Software

```

```

        Solutions"));
11     System.out.println("compareTo(\"Java\") : " + title.compareTo("Java"));
12     String hello = "Hello";
13     System.out.println("last char index : " + (hello.length() - 1));
14     System.out.println("hello.substring(1,4) : " + hello.substring(1, 4));
15     System.out.println("hello.substring(5) : [" + hello.substring(5) + "]");
16
17     // strings are immutable
18     String s = "cat";
19     s.toUpperCase();
20     System.out.println("after s.toUpperCase(); s is still " + s);
21     s = s.toUpperCase();
22     System.out.println("after s = s.toUpperCase(); s is " + s);
23 }
24 }

```

Output

```

length : 23
substring(5) : Software Solutions
substring(0, 4) : Java
indexOf("Soft") : 5
indexOf("zzz") : -1
indexOf("o") : 6
equals : true
compareTo("Java") : 19
last char index : 4
hello.substring(1,4) : ell
hello.substring(5) : []
after s.toUpperCase(); s is still cat
after s = s.toUpperCase(); s is CAT

```

Check the numbers against the diagram idea. "Java Software Solutions" has $4+1+8+1+9 = 23$ characters. `indexOf("Soft")` is 5 because the word Software starts after "Java " (indexes 0–4). `indexOf("o")` is 6: the first lower-case o is the 'o' of Software. `compareTo("Java")` is 19 since "Java" is a prefix of the title: the answer is the difference in lengths, $23 - 4 = 19$. Last, `hello.substring(5)` is the empty string `""`: legal, because index 5 equals the length.

★ valid index ranges

For a string of length n : `charAt(i)` needs $0 \leq i \leq n - 1$; `substring(a, b)` needs $0 \leq a \leq b \leq n$; `substring(a)` needs $0 \leq a \leq n$. Violating any of these throws `StringIndexOutOfBoundsException`.

11.1.4 Strings are immutable

A `String` object can never be changed after it is created. Methods like `toUpperCase()` do *not* modify the string they are called on; they **return a new string**. If you ignore the returned value, nothing happens:

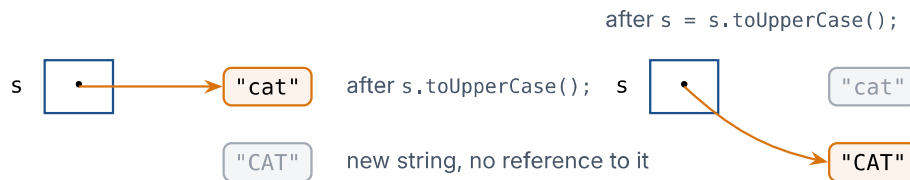


Figure 11.2: Immutability. `s.toUpperCase();` builds a new string `"CAT"` but nothing refers to it, so `s` still refers to `"cat"`. Only the assignment `s = s.toUpperCase();` makes `s` point to the new string.

The last two lines of the output above prove it. `StringMutation` chains several operations, each producing a new string from the previous one:

StringMutation.java

```
1 // StringMutation.java
2 // Demonstrates the use of the String class and its methods.
3
4 public class StringMutation {
5     public static void main(String[] args) {
6         String phrase = "Change is inevitable";
7         String mutation1, mutation2, mutation3, mutation4;
8
9         System.out.println("Original string: \"" + phrase + "\"");
10        System.out.println("Length of string: " + phrase.length());
11
12        mutation1 = phrase.concat(", except from vending machines.");
13        mutation2 = mutation1.toUpperCase();
14        mutation3 = mutation2.replace('E', 'X');
15        mutation4 = mutation3.substring(3, 30);
16
17        // Print each mutated string
18        System.out.println("Mutation #1: " + mutation1);
19        System.out.println("Mutation #2: " + mutation2);
20        System.out.println("Mutation #3: " + mutation3);
21        System.out.println("Mutation #4: " + mutation4);
22
23        System.out.println("Mutated length: " + mutation4.length());
24    }
25 }
```

Output

```
Original string: "Change is inevitable"
Length of string: 20
Mutation #1: Change is inevitable, except from vending machines.
Mutation #2: CHANGE IS INEVITABLE, EXCEPT FROM VENDING MACHINES.
Mutation #3: CHANGX IS INXVITABLX, XXCXPT FROM VXNDING MACHINXS.
Mutation #4: NGX IS INXVITABLX, XXCXPT F
Mutated length: 27
```

Follow it: `mutation1` appends text; `mutation2` upper-cases; `mutation3` replaces `E` by `X`; `mutation4` keeps indexes 3 up to (not including) 30, which is $30 - 3 = 27$ characters (the last line). The length of the result of `substring(a, b)` is always $b - a$.

⇌ From Python to Java: immutable strings

Python strings are immutable too, and `s.upper()` also returns a new string. Java behaves the same, but unlike Python it won't let you write `s[0] = "X"` at all (there is no indexing assignment on strings).

Checkpoint 11.1

Predict the four lines printed:

StringQuick.java

```
1 public class StringQuick {
2     public static void main(String[] args) {
3         String str = "Space, the final frontier.";
4         System.out.println(str.length());
5         System.out.println(str.substring(7));
6         System.out.println(str.toUpperCase());
7         System.out.println(str.length());
8     }
9 }
```

11.1.5 When string methods fail

Asking for positions that do not exist crashes the program:

StringErrors.java

```
1 public class StringErrors {
2     public static void main(String[] args) {
3         String s = "Hello";
4         System.out.println(s.substring(2, 3));
5         System.out.println(s.substring(3, 2));
6     }
7 }
```

Output

```
l
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: Range [3, 2) out of
    bounds for length 5
    at java.base/java.lang.String.substring(String.java:2823)
    at StringErrors.main(StringErrors.java:5)
```

(The real message continues with several lines about the inside of the Java library; read downward until you reach a line mentioning *your* class and line number — here `StringErrors.java:5`.) `s.substring(2, 3)` is valid and gives `"l"`; `s.substring(3, 2)` asks for a start after the end.